
Task W02: Backtracking

Week 2 (Choose, Explore, Backtrack)

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Important Guidelines

- Exam questions and problems may extend beyond tasks and lab assignments.
- No Generative AI: You must complete this assignment using only the resources provided in the course (book, lectures, and materials). Credit will not be granted for using external features or techniques not covered in this course. (See the one exception in the Eight-Queens section.)
- Ask Me Questions: If you need clarification, please ask me directly. Other instructors or advisors may not be aware of the specific details and expectations of this assignment.
- The assignment instructions do not detail every step; therefore, you must read, interpret, and possibly make assumptions. If something is unclear, write down the assumption you make, proceed with it, and complete the problem. Do not leave a problem unfinished because a detail was ambiguous. Not all assumptions are automatically correct, so state yours clearly enough that I can follow your reasoning.

2 How to Submit Your Work

Everything for this assignment goes on your answer sheet: the boards for Problems 1 and 2, your backtrack notes, your rewritten pseudocode, and your written answers for Problem 3. There is nothing to draw or submit separately.

If you find something that needs to be debugged

If, while working through this assignment, you discover something in the problem statements that needs to be debugged, describe what you found in the "Bugs or Issues You Found" section of the answer sheet.

Where to submit

Commit your full submission, including your answer sheet's `.tex` source, its compiled PDF, and your AI conversation if you used the formatting exception in the Eight-Queens section, to a folder

named `taskw02` in your git repository.

3 How These Problems Work

A backtracking search follows the same three-part pattern every time: choose an option, explore what happens if you take it, and if that exploration cannot succeed, undo the choice and backtrack to try the next option. This is the same calls-and-returns idea from the recursion basics assignment, except a call here can have several options to try instead of one, and some of those options fail before reaching a base case.

For Problems 1 and 2, you will not diagram the entire search. Instead you will fill in the board on your answer sheet with the algorithm's final result: queen positions for Eight-Queens. Where a problem asks for backtrack notes, record them in the Backtrack Notes section of your answer sheet; the point is to show you actually worked through the algorithm rather than guessed at an answer, without requiring a full diagram of every call. Problem 3 takes a different approach, using a video instead of hand execution; see Section 5 for details.

4 Eight-Queens

Eight-Queens asks whether N queens can be placed on an $N \times N$ board so that no two queens share a row, a column, or a diagonal. The book's version uses an eight by eight board, which is far too large to work through reliably by hand. Both problems below use a four by four board instead, small enough to work through completely.

The pseudocode below places one queen per column, choosing which row to try it in, and stops as soon as one full solution is found.

```
function SolveQueensByColumn(column):
    if column == N:
        return true // every column has a queen; solved
    for row = 0 to N - 1:
        if placing a queen at (row, column) is safe, given the
           queens already placed in earlier columns:
            place a queen at (row, column)
            if SolveQueensByColumn(column + 1):
                return true // solution found; stop searching
            remove the queen at (row, column) // backtrack
    return false // dead end: no row worked here
```

4.1 Problem 1: A Smaller Board

Using the pseudocode above exactly as written, work through `SolveQueensByColumn` on a four by four board, starting from column 0. Fill in the board on your answer sheet, labeled with columns 1 through 4 left to right and rows 1 through 4 top to bottom, and place a queen in each square where the algorithm's first full solution ends up.

For any column where the row you tried first did not work, add one short note in the Backtrack Notes section of your answer sheet: which column, which row you tried first, and why it failed.

4.2 Problem 2: Row by Row

Before you work through anything, rewrite the pseudocode above so that it places one queen per row instead of one queen per column, choosing which column to try the queen in for each row.

Write your rewritten pseudocode in the space provided on your answer sheet for this problem.

Then work through your rewritten version on the same four by four board, starting from row 0, and fill in the board on your answer sheet the same way you did for Problem 1, including one short note in the Backtrack Notes section for any row where your first-tried column did not work.

Once you have solved this problem, and only after you have solved it, you may use an AI tool solely to format your finished pseudocode. If you do this, you must explicitly tell the AI not to change or fix your algorithm, only to format it, and you must submit the full AI conversation alongside your pseudocode. This exception does not extend to using AI to solve the problem, or to fix a mistake in your algorithm; that is still covered by the No Generative AI guideline above.

5 Knight's Tour

A knight's tour asks whether a knight can visit every square on a board exactly once, moving only in the knight's usual L-shape, never revisiting a square. Unlike Eight-Queens, a single call in this search has up to eight options to try, since a knight can move in up to eight directions from any square, fewer near an edge or a corner.

The pseudocode below tries a knight's eight possible moves in a fixed order every time, and stops as soon as the entire board has been visited.

```
function SolveKnightsTour(row, col, squaresVisitedSoFar):
    mark (row, col) as visited
    if squaresVisitedSoFar == totalSquares:
        return true // every square visited; solved

    rowOffset = [-2, -2, -1, -1, 1, 1, 2, 2]
    colOffset = [-1, 1, -2, 2, -2, 2, -1, 1]

    for i = 0 to 7:
        nextRow = row + rowOffset[i]
        nextCol = col + colOffset[i]
        if 0 <= nextRow < numRows and 0 <= nextCol < numCols
            and (nextRow, nextCol) is not visited:
                if SolveKnightsTour(nextRow, nextCol, squaresVisitedSoFar + 1):
                    return true // solution found; stop searching

    mark (row, col) as unvisited // backtrack: undo this square so
    return false // the caller can try its next move
```

5.1 Worked Example

Here is what `SolveKnightsTour` finds on a four row by three column board (twelve squares), starting from square B1 (column B, row 1). Columns run A, B, C left to right; rows run 1 through 4 top to bottom. The number in each square below is the order the knight visited it.

Reaching this board was not a straight line. It took 31 total recursive calls and 19 backtracks to get here. For example, after ten confident moves in a row, square C4 turned out to be a dead end: every neighboring square was already visited. The search had to backtrack through four earlier squares, C4, B2, A4, and C3, before a new path forward finally opened up at A2's next candidate move. This is exactly the choose, explore, backtrack pattern from Section 3; it is just not diagrammed here move by move.

Notice that this small twelve-square board already needed 31 calls, nearly twice the number of

Table 1: Knight's Tour worked example: visit order starting from B1

	A	B	C
1	10	1	8
2	7	4	11
3	2	9	6
4	5	12	3

squares on the board. Keep that in mind for the reflection questions.

5.2 Problem 3: Watch and Answer

Watch this video on the Knight's Tour puzzle:

Knight's Tour — Numberphile
https://www.youtube.com/watch?v=ab_dY3dZFHM

As you watch, notice how many possible paths the knight has to consider, and how much of solving this by hand really amounts to organized trial and error, the same choose, explore, backtrack pattern from Section 3.

Then answer the following in a few sentences each on your answer sheet.

1. In your own words, explain what backtracking means for a knight's tour search. Use the words "dead end" and "undo" somewhere in your answer.
2. Based on what you saw with Eight-Queens, would you expect a backtracking knight's tour search starting from a corner square to need more backtracking, less, or is it impossible to say without trying, compared to starting from a square near the middle of the board? Explain your reasoning, thinking about how many legal moves a knight has from each type of square.
3. Suppose the fixed order in which the eight knight moves are tried, given in the pseudocode above, was changed to a different fixed order, similar to how Problem 2 changed Eight-Queens from column by column to row by row. Would you expect a valid tour to still turn up just as quickly, or could the order matter here too? Explain your reasoning.
4. A three by three board has a center square that a knight can never move to or from anywhere else on that board. Based on this, could a backtracking search ever find a complete tour on a three by three board, no matter where it starts or what order it tries moves in? Explain your reasoning.

6 Reflection Questions

Answer these in a few sentences each, after you have completed your work above.

1. Compare your Problem 1 board to your Problem 2 board, along with the backtrack notes you wrote for each. Are the boards the same, mirrored, or something else entirely? Did one problem need more backtrack notes than the other? Explain why, based on the shape of the board and how the safety check works.
2. The Knight's Tour worked example took 31 total calls and 19 backtracks on a twelve-square board. Your Problem 1 board has sixteen squares, more than the knight's board, yet almost

certainly needed far fewer backtrack notes. What is different about the two problems that could explain this?

3. Look back at your answer to Problem 3, question 2, about corner versus central starting squares. Does that reasoning match what you actually saw happen across your Eight-Queens work, where changing where the search starts (column 0 for Problem 1, row 0 for Problem 2) still ended up exploring a similar amount? Explain any similarities or differences you notice.
4. Problem 2 asked you to change Eight-Queens from column by column to row by row. Suppose you instead kept the queens problem exactly as written for Problem 1, and only changed the order that rows are tried within each column, for example trying the bottom row first instead of the top row. Based on what you saw across Problems 1 through 3, would you expect that change to behave more like Problem 2, or more like the difference you saw between Problem 1 and the Knight's Tour example? Explain your reasoning.